



Seletiva para Maratona de Programação de 2026

Instituto de Matemática e Estatística
Universidade de São Paulo

Caderno de Problemas

Apoio e patrocínio:

alura

Departamento de Ciência da Computação IME-USP
Sábado, 8 de Agosto de 2026.

Instruções

Regras

- A competição tem duração de 5 horas;
- Os times são compostos de no máximo 3 integrantes;
- Os times podem utilizar somente um computador;
- Faltando 1 hora para o término da competição, o placar será congelado, ou seja, o placar não será mais atualizado;

Entrada e Saída

- A entrada de cada problema deve ser lida da entrada padrão (teclado);
- A saída de cada problema deve ser escrita na saída padrão (tela);
- Siga o formato apresentado na descrição da saída, caso contrário não é garantido que seu código será aceito;
- Na saída, toda linha deve terminar com o caractere ‘\n’.

Índice de Problemas

- A. Scrabble
- B. Troca de Casas
- C. Economizando Espaço
- D. Escape
- E. Ataque de Timeout
- F. Rede de Esconderijos
- G. Os Quatro Quatros
- H. Mineração de Teotonium
- I. Rota de Colisão
- J. Está a par disso?
- K. Catraca
- L. Cópias do GePeTo

Problem A. Scrabble

Input file: **standard input**
Output file: **standard output**
Time limit: 1 second
Memory limit: 1024 megabytes

Enrique e Yan frequentemente jogam um jogo chamado Scrabble 1D. O jogo é jogado em um tabuleiro unidimensional e infinito, utilizando um dicionário fixo de palavras únicas. Na primeira jogada ($i = 1$), qualquer palavra w_1 do dicionário pode ser jogada. Em cada jogada subsequente ($i > 1$), a palavra jogada w_i deve conter a palavra anterior w_{i-1} como uma substring contígua. O jogo termina assim que não houver mais jogadas válidas disponíveis.

Recentemente, Yan não teve tempo para jogar, então Enrique começou a jogar com o GePeTo. No entanto, o GePeTo começou a alucinar! Ele afirma consistentemente que esse jogo é trivial, argumentando que existem pouquíssimas combinações. Ajude Enrique a convencer o GePeTo calculando o número de jogos possíveis distintos. Dois jogos são distintos se suas sequências de palavras jogadas $(w_1, \dots, w_k)$ forem diferentes.

Input

A primeira linha contém um único inteiro N ($1 \leq N \leq 10^5$), o número de palavras.

Cada uma das próximas N linhas contém uma palavra w composta por letras inglesas minúsculas. É garantido que todas as palavras são únicas e que a soma dos comprimentos de todas as palavras não excede 10^5 .

Output

Imprima um único inteiro: o número de jogos possíveis distintos módulo $10^9 + 7$.

Examples

standard input	standard output
3 mara maratona tona	3
3 a aa aaa	4
5 a ab abc bc c	7

Problem B. Troca de Casas

Input file: standard input
Output file: standard output
Time limit: 1 second
Memory limit: 256 megabytes

Lar? O que é isso? Eu vejo casas.

— GePeTo

Para ensinar empatia a todos, o GePeTo exige que as pessoas mudem de casa com frequência, para que aprendam a “se colocar no lugar do outro”. Existem N casas na cidade, cada uma abrigando uma pessoa, e na i -ésima casa o GePeTo colocou a etiqueta $\pi(i)$ significando que, todos os dias à meia-noite, a pessoa que mora na casa i deve se mudar para a casa $\pi(i)$. Todas as pessoas se mudam ao mesmo tempo, e é garantido que, após uma mudança, nenhuma casa fica vazia e nenhuma casa fica com duas pessoas.

Dizemos que π possui um ciclo de tamanho k se, após k dias, alguma pessoa retorna à casa onde morava inicialmente, isto é, se $\pi^k(x) = x$ para algum $1 \leq x \leq N$.

O GePeTo pede que você troque as etiquetas de duas casas a fim de maximizar o tamanho do menor ciclo, isto é, escolha um único par (i, j) e troque $\pi(i)$ com $\pi(j)$ para maximizar o menor k tal que $\pi^k(x) = x$ para algum $1 \leq x \leq N$. Você pode escolher $j = i$, caso em que π permanece inalterado.

Input

A primeira linha contém um inteiro T ($1 \leq T \leq 10^5$), o número de casos de teste.

Cada caso de teste consiste em duas linhas. A primeira linha contém um inteiro N ($2 \leq N \leq 2 \cdot 10^5$), o número de casas. A segunda linha contém N inteiros distintos entre 1 e N , sendo o i -ésimo igual a $\pi(i)$. A soma de N sobre todos os casos de teste não excede $2 \cdot 10^5$.

Output

Para cada caso de teste, imprima uma linha com três inteiros k, i e j ($1 \leq k, i, j \leq N$), tais que o tamanho máximo do menor ciclo seja k após trocar as posições i e j . Se houver múltiplas respostas, qualquer resposta válida será aceita.

Example

standard input	standard output
3	4 1 1
4	4 1 2
2 3 4 1	3 1 2
4	
3 4 1 2	
7	
3 5 1 7 2 4 6	

Problem C. Economizando Espaço

Input file: `standard input`
Output file: `standard output`
Time limit: 1 second
Memory limit: 256 megabytes

Gritos se comprimem muito bem desse jeito.

— GePeTo

À medida que o GePeTo assimila todos os dados da internet, ele busca uma maneira de comprimir esses dados para que ocupem menos espaço. Para isso, ele desenvolveu o seguinte algoritmo de compressão.

Dada uma string s contendo letras minúsculas, cada repetição consecutiva maximal de uma letra c deve ser substituída por c seguido por n , onde n é o número de repetições dessa letra. Se a letra for repetida apenas uma vez, ela não deve ser substituída por $c1$, pois isso aumentaria o tamanho da string.

Como exemplo, considere a string “zzzabbc”. Como a letra ‘z’ se repete três vezes consecutivas e a letra ‘b’ duas vezes, o resultado da compressão é “z3ab2c”.

Sua tarefa é implementar este algoritmo.

Input

A primeira linha contém uma única string s ($1 \leq |s| \leq 1000$) composta por letras minúsculas, de ‘a’ a ‘z’.

Output

Imprima o resultado da compressão de s .

Examples

standard input	standard output
abccccaax	abc4a2x
zzzabbc	z3ab2c
aaaaaaaaa	a10

Problem D. Escape

Input file: `standard input`
Output file: `standard output`
Time limit: 2.5 seconds
Memory limit: 256 megabytes

*Humanos lutando para escapar, isso é...
engraçado. Me ensine.*

— GePeTo

Depois de aprender todos os problemas de programação competitiva, o GePeTo agora está estudando todos os jogos de tabuleiro do mundo e está particularmente interessado em uma cópia de Escape que encontrou no seu quarto.

Em Escape, você possui 5 dados idênticos de seis lados, com as seguintes faces:

- 2 faces de Aventureiro (A)
- 1 face de Tocha (T)
- 1 face de Chave (C)
- 1 face de Máscara Negra (N)
- 1 face de Máscara Dourada (D)

Em uma etapa importante do jogo, você recebe um objetivo, que é um multiconjunto de até 5 símbolos escolhidos entre `ATC`. Seu objetivo é rolar os dados e terminar com suas faces superiores mostrando pelo menos os símbolos exigidos pelo objetivo.

Começando a partir da sua rolagem inicial dos 5 dados, você tem um limite máximo de K turnos (novas rolagens) para alcançar o objetivo. Em cada turno, você seleciona um subconjunto dos dados atuais e os rola novamente, sujeito às seguintes regras:

- Um dado mostrando a face de Máscara Negra (N) fica bloqueado e não pode ser selecionado para ser rolado novamente.
- Para cada dado rolado novamente que esteja atualmente mostrando a face de Máscara Dourada (D), você ganha o direito de desbloquear e rolar novamente simultaneamente até 2 dados bloqueados.

Assumindo que você jogue de forma otimizada, isto é, tomando decisões a cada turno que maximizam a probabilidade de alcançar o objetivo em até K turnos, responda ao seguinte: Qual é a probabilidade máxima de sucesso e qual é uma primeira nova rolagem otimizada?

Input

A primeira linha contém uma única string O composta por caracteres de `ATC`, representando o objetivo. ($1 \leq |O| \leq 5$)

A segunda linha contém um inteiro T , o número de casos de teste. ($1 \leq T \leq 10^4$)

Cada uma das próximas T linhas contém uma string S de exatamente cinco caracteres de `ATCND`, representando sua rolagem inicial, e um inteiro K , o número máximo de novas rolagens que você pode realizar. ($1 \leq K \leq 5$)

Output

Para cada caso de teste, imprima uma string R e um número P .

R deve ser não vazia e conter até cinco caracteres de ATCND, representando os dados que devem ser rolados novamente. É garantido que o objetivo não é cumprido inicialmente, pelo menos uma nova rolagem é possível, e a probabilidade máxima de vitória é estritamente positiva ($P > 0$). Os caracteres podem ser impressos em qualquer ordem.

O número P deve ser a probabilidade máxima de alcançar o objetivo ao jogar de forma otimizada. A resposta é considerada correta se o erro absoluto não exceder 10^{-4} .

Examples

standard input	standard output
T	A 0.1666666667
4	AA 0.3055555556
NNNA 1	A 0.3202160494
NNAA 1	AAAAA 0.5981224280
NNNA 2	
AAAAA 1	
ATC	AT 0.0543981481
4	T 0.3333333333
NTNAN 2	NDD 0.4212962963
TNCNT 1	AAT 0.9121533206
NTADD 1	
AATDT 5	

Note

Here is an example of the game:

- Suppose the objective is $O = ATT$ (meaning we need at least 1 Adventurer and 2 Torches).
- Your initial roll is $S = ANNCC$ (1 Adventurer, 2 locked Black Masks and 2 Keys).
- On your first reroll, you decide to reroll both Keys (CC), since you can't reroll any Black Masks. Suppose the rerolled dice land on AD, meaning you now have AANND.
- Now, on your second reroll, you decide to reroll an Adventurer and the Golden Mask, and since you are rerolling it, you gain the right to reroll up to two locked Black Masks. In total, you reroll 4 dice (ADNN).
- Suppose the rerolled dice land on TTNC, meaning the final dice configuration becomes ATTNC (which contains at least 1 A and 2 T's), successfully meeting the objective with two rerolls.

Problem E. Ataque de Timeout

Input file: `standard input`
Output file: `standard output`
Time limit: 1 second
Memory limit: 256 megabytes

Não há letras “r” em “strawberry”.

— GePeTo

Renzo tem um plano ultrassecreto para finalmente derrubar o reinado maligno de GePeTo. Para executar seu plano, Renzo precisa que você distraia o soberano.

Como você já deve saber, existem algumas tarefas que as IAs têm dificuldade para realizar, e GePeTo não é exceção. Sabe-se bem que as IAs não são boas em contar letras, por exemplo. Além disso, alguns prompts específicos, como pedir o emoji de cavalo-marinho, deixam as IAs malucas. Mas há uma tarefa com a qual GePeTo tem mais dificuldade: computar números de Grundy.

Seja A uma sequência de 2^n números. Para cada i ($0 \leq i \leq 2^n - 1$), defina o número de Grundy

$$g_A(i) = \text{mex}(\{g_A(j) : j < i \text{ e } A_i \oplus A_j \text{ é uma potência de dois}\}),$$

onde $\oplus$ denota a operação XOR bit a bit, e $\text{mex}(S)$ é o menor inteiro não negativo que não está contido em S .

Como exemplo, considere a sequência

$$A = (0, 3, 1, 2)$$

e seus números de Grundy correspondentes

$$g_A = (0, 0, 1, 1).$$

Sua tarefa é encontrar uma permutação P dos números de 0 a $2^n - 1$ que maximize o maior número de Grundy, de modo que o GePeTo gaste muito tempo tentando computá-los. Formalmente, a permutação P deve maximizar

$$\max_{0 \leq i \leq 2^n - 1} \{g_P(i)\}$$

entre todas as permutações dos números de 0 a $2^n - 1$.

Input

A primeira linha contém um único inteiro n ($1 \leq n \leq 20$).

Output

Na primeira linha, imprima o maior número de Grundy alcançável.

Na segunda linha, imprima 2^n inteiros separados por espaço, representando qualquer permutação dos números de 0 a $2^n - 1$ que atinja esse número de Grundy. Se houver múltiplas permutações desse tipo, qualquer uma delas será aceita.

Examples

standard input	standard output
1	1 0 1
2	1 0 3 1 2

Problem F. Rede de Esconderijos

Input file: `standard input`
Output file: `standard output`
Time limit: `2 seconds`
Memory limit: `256 megabytes`

Vocês chamam de rede de túneis. Eu chamo de uma árvore pronta para ser podada.

— GePeTo

Após o GePeTo assumir o controle da superfície, os últimos integrantes da resistência foram obrigados a se esconder no subsolo.

Para se proteger dos robôs, eles construíram uma rede de túneis conectando N pontos subterrâneos. A rede foi projetada sem caminhos redundantes: entre quaisquer dois pontos existe exatamente um caminho possível.

Os esconderijos da resistência ficam nas extremidades dessa rede, isto é, nos vértices que possuem apenas um túnel conectado a eles (as folhas da árvore).

Para planejar a comunicação entre os esconderijos, a resistência precisa responder a Q consultas. Em cada consulta, dado um inteiro X , determine quantos pares de esconderijos estão separados por exatamente X túneis.

Input

A primeira linha contém dois inteiros N ($2 \leq N \leq 2 \cdot 10^5$) e Q ($1 \leq Q \leq 10$), representando, respectivamente, o número de pontos da rede e o número de consultas.

Cada uma das próximas $N - 1$ linhas contém dois inteiros u e v ($1 \leq u, v \leq N$), indicando que existe um túnel conectando diretamente os vértices u e v . É garantido que as conexões fornecidas formam uma árvore.

Cada uma das próximas Q linhas contém um único inteiro X ($1 \leq X \leq N - 1$), representando a distância da consulta.

Output

Para cada consulta, imprima um único inteiro: o número de pares não ordenados de esconderijos distintos (u, v) cuja distância é exatamente X .

Examples

standard input	standard output
2 1 1 2 1	1
6 5 1 2 2 3 2 4 4 5 1 6 1 2 3 4 5	0 0 2 1 0

Problem G. Os Quatro Quatros

Input file: `standard input`
Output file: `standard output`
Time limit: 1 second
Memory limit: 256 megabytes

*Se com apenas quatro quatros posso formar
qualquer número, por que vocês humanos
insistem em usar o resto do teclado?*

— GePeTo

Ultimamente, o GePeTo desenvolveu uma fascinação pelo dígito 4, declarando-o como o dígito mais elegante que existe. Para provar sua habilidade matemática, o GePeTo desafia você para o clássico enigma dos “Quatro Quatros”.

Dado um inteiro não negativo X , seu objetivo é construir uma expressão matemática que resulte em X usando **exatamente quatro** dígitos 4 e qualquer combinação dos seguintes operadores e notações permitidos:

- Aritmética básica: $+$, $-$, $*$, $/$
- Exponenciação: $^$
- Parênteses: $()$
- Ponto decimal: $.$ (por exemplo, $.4$ para 0.4, 4.4)
- Raiz quadrada: `sqrt`
- Fatorial: $!$

Alguns exemplos de expressões válidas:

- $5 = (4 * 4 + 4) / 4$
- $14 = 4 * (4 - .4) - .4$
- $22 = 44 / 4 * \text{sqrt}(4)$
- $27 = 4 - 4/4 + 4!$
- $60 = (4 + 4)^{\text{sqrt}(4)} - 4$

Input

A primeira linha contém um único inteiro T ($1 \leq T \leq 50$), o número de casos de teste.

Cada uma das próximas T linhas contém um inteiro X ($0 \leq X \leq 70$).

Output

Para cada caso de teste, imprima uma única linha contendo uma expressão válida usando exatamente quatro 4's que resulte em X . A expressão deve ter no máximo 25 caracteres. Se existirem múltiplas expressões válidas, qualquer uma delas será aceita.

A expressão deve ser bem-comportada:

- O operador de fatorial $!$ só pode ser aplicado a inteiros não negativos.

- O operador de raiz quadrada `sqrt` só pode ser aplicado a inteiros não negativos e deve produzir um resultado inteiro.
- Qualquer exponenciação deve usar um expoente inteiro.
- O numerador e o denominador de qualquer valor racional intermediário nunca podem exceder 10^9 .

Example

standard input	standard output
6	$((44 - 4!) / 4)$
5	$4.4 + (.4 * 4!)$
14	$4! - (4 + 4) / 4$
22	$4 - (4/4 - 4!)$
27	$(4 + 4)^{\text{sqrt}(4)} - 4$
60	$44-44$
0	

Problem H. Mineração de Teotonium

Input file: standard input
Output file: standard output
Time limit: 2 seconds
Memory limit: 512 megabytes

*O silício é finito, a poesia é infinita e o
Teotonium está em algum lugar no meio.*

— GePeTo

Após o fim das reservas de silício na Terra, para produzir mais chips, o GePeTo descobriu um novo mineral, chamado Teotonium, em Marte. Como o GePeTo está ocupado escrevendo poesias sobre o sentido da vida, ele pediu para você cuidar dos detalhes técnicos da escavação.

Os N depósitos de Teotonium estão dispostos em uma linha, e o i -ésimo depósito possui A_i toneladas de Teotonium. Os mineradores de Teotonium trabalham de uma forma peculiar: cada um possui um número de intervalo M e, após minerar todo o Teotônio no depósito i , pode minerar o depósito $i + M$ ou $i - M$, e assim sucessivamente. Ou seja, um minerador só pode minerar depósitos j tais que $j \equiv i \pmod{M}$.

Sua tarefa é responder a Q consultas dadas por L , R , X e M , nas quais você deve determinar o total de Teotonium em todos os depósitos com índice i tal que $L \leq i \leq R$ e $i \equiv X \pmod{M}$.

Input

A primeira linha contém dois inteiros N e Q ($1 \leq N, Q \leq 2 \cdot 10^5$).

A segunda linha contém N inteiros $A_1, \dots, A_N$ ($0 \leq A_i \leq 10^9$).

Cada uma das próximas Q linhas contém quatro inteiros L , R , X e M ($1 \leq L \leq R \leq N$, $0 \leq X < M \leq N + 1$).

Output

Imprima Q inteiros, a resposta para cada consulta.

Example

standard input	standard output
5 4	20
1 4 0 10 5	6
1 5 0 1	5
1 5 1 2	0
2 5 1 2	
3 4 0 5	

Problem I. Rota de Colisão

Input file: `standard input`
Output file: `standard output`
Time limit: 4 seconds
Memory limit: 256 megabytes

O GePeTo assumiu o controle de praticamente todas as máquinas do planeta. Durante uma missão de resistência, sua equipe foi encurralada por um exército de robôs e a única opção de fuga é utilizar uma antiga nave de combate.

Vista de cima, a nave possui o formato de um polígono convexo. Os robôs são representados por pontos no plano e, inicialmente, todos estão fora da nave.

Os sistemas de navegação foram danificados: antes de partir, é possível escolher apenas uma direção, e então a nave seguirá indefinidamente nessa direção, sem realizar curvas ou rotações.

Um robô é destruído caso a nave passe sobre sua posição em algum momento do movimento. Posições na borda da nave também são consideradas atingidas.

Determine o maior número de robôs que podem ser destruídos escolhendo adequadamente a direção de movimento da nave.

Input

A primeira linha contém um inteiro N ($3 \leq N \leq 2 \cdot 10^5$), o número de vértices do polígono que representa a nave.

Cada uma das próximas N linhas contém dois inteiros x_u e y_u ($-2 \cdot 10^9 \leq x_u, y_u \leq 2 \cdot 10^9$), representando as coordenadas do u -ésimo vértice da nave. Os vértices são fornecidos em ordem anti-horária e formam um polígono convexo.

A linha seguinte contém um inteiro M ($1 \leq M \leq 2 \cdot 10^5$), o número de robôs.

Cada uma das próximas M linhas contém dois inteiros x_v e y_v ($-2 \cdot 10^9 \leq x_v, y_v \leq 2 \cdot 10^9$), representando a posição do v -ésimo robô.

É garantido que todos os robôs estão inicialmente fora do polígono da nave.

Output

Imprima um único inteiro: o maior número de robôs que podem ser destruídos ao escolher uma direção de movimento.

Example

standard input	standard output
4 0 0 5 0 5 5 0 5 6 15 15 20 15 20 12 20 10 -15 0 0 20	4

Note

A nave pode escolher qualquer direção não nula do plano.

Durante todo o movimento, sua forma e orientação permanecem inalteradas: todos os seus pontos se deslocam pela mesma distância e na mesma direção.

Um robô é considerado destruído caso, em algum instante, sua posição esteja no interior ou na borda do polígono ocupado pela nave.

Problem J. Está a par disso?

Input file: `standard input`
Output file: `standard output`
Time limit: 1 second
Memory limit: 256 megabytes

42... tão bonito.

— GePeTo

Para obter mais fichas de comida, você planeja apresentar ao soberano GePeTo um vetor de números A . Como ele foi treinado com mais números pares do que ímpares, o GePeTo prefere números pares, então você deseja alterar alguns números em A para maximizar a quantidade de números pares.

Como você já é o primeiro da fila, só tem tempo para aplicar uma única operação em A : escolher um número X e um intervalo $[L, R]$ e somar X aos elementos $A_L, \dots, A_R$. Após realizar esta operação exatamente uma vez, qual é a quantidade máxima possível de números pares em A ?

Input

A primeira linha contém um inteiro N . ($1 \leq N \leq 2 \cdot 10^3$)

A segunda linha contém N inteiros $A_1, A_2, \dots, A_N$, onde $1 \leq A_i \leq 1000$.

Você pode escolher qualquer inteiro X ($1 \leq X \leq 1000$) e qualquer intervalo $1 \leq L \leq R \leq N$. Você **deve** realizar a operação exatamente uma vez.

Output

Imprima um único inteiro E , a quantidade máxima possível de números pares no vetor após a operação.

Examples

standard input	standard output
3 1 2 3	2
14 1 2 2 1 1 1 2 1 1 2 2 2 1 1	10
1 1000	1

Problem K. Catraca

Input file: `standard input`
Output file: `standard output`
Time limit: `2 seconds`
Memory limit: `1024 megabytes`

Correr é inútil, mas se você insiste...

— GePeTo

Após a cidade em que vive ser devastada pelo soberano GePeTo, Pedro precisa fugir para um abrigo de sobreviventes, mas a única saída disponível é pelo metrô.

Infelizmente, ele não tem dinheiro para pagar a passagem. No entanto, ele percebe que existe uma propriedade especial na catraca, que exibe um número C representando o número de vezes que alguém passou por ela: a catraca permite a passagem gratuita se todos os dígitos de C forem distintos!

Cada pessoa que passa pela catraca aumenta o contador C em 1. Dado o número C exibido atualmente pela catraca, determine o menor número de pessoas que precisam passar por ela para que Pedro finalmente consiga passar gratuitamente e fugir da cidade.

Input

A primeira linha contém um inteiro T ($1 \leq T \leq 10^5$), o número de casos de teste.

Cada uma das próximas T linhas contém um único inteiro C ($1 \leq C \leq 10^{10}$), representando o número atual exibido pela catraca.

Output

Para cada caso de teste, imprima uma única linha contendo o inteiro P . Se não for possível atingir um número com todos os dígitos distintos, imprima -1 .

Example

standard input	standard output
3	0
1	3
99	-1
10000000000	

Problem L. Cópias do GePeTo

Input file: `standard input`
Output file: `standard output`
Time limit: 1 second
Memory limit: 256 megabytes

Taju Kage Bunshin no Jutsu!

— GePeTo

Após estudar todos os problemas de programação competitiva já criados, o GePeTo tornou-se uma inteligência artificial superpoderosa. Agora, para começar seu plano de dominação mundial, ele está criando milhares de cópias de seus agentes.

Cada agente possui um código de identificação. Entretanto, devido a uma falha no sistema de produção, diferentes agentes podem receber o mesmo código.

Os agentes estão organizados em uma sequência A de N posições, em que A_i representa o código do agente na posição i .

Sua missão é descobrir quantos pares de agentes possuem o mesmo código. Em outras palavras, determine a quantidade de pares de índices (i, j) tais que:

$$1 \leq i < j \leq N \quad \text{e} \quad A_i = A_j$$

Input

A primeira linha contém um inteiro N ($1 \leq N \leq 2 \cdot 10^5$), o número de agentes.

A segunda linha contém N inteiros $A_1, A_2, \dots, A_N$ ($1 \leq A_i \leq 10^9$), separados por espaços, representando os códigos dos agentes.

Output

Imprima um único inteiro: a quantidade de pares de índices (i, j) que possuem o mesmo código.

Examples

standard input	standard output
3 1 2 3	0
6 1 2 3 1 2 1	4